

Exploring Python for IoT

Python is a widely-used and versatile programming language with applications spanning various domains, including the Internet of Things (IoT). One of the devices that can be used for IoT projects is the Raspberry Pi Pico, as this low-cost microcontroller board can run Python code. To program the Pico with Python, we will use CircuitPython, a Python variant optimised for microcontrollers and embedded systems.

CircuitPython has many built-in libraries and modules that simplify the coding and interfacing with hardware components. To power our Pico device, we will need a power source that can provide 5V and 2A current source. Options include a battery pack or a power adapter with a micro-USB connector. For this article, I have used an Adafruit adapter, which is a USB wall charger compatible with the Pico board. You can also use any other power adapter that meets the specifications and has a micro-USB port.

Circuit connections

Here is a sample of a digital circuit that I created. It connects GPIO pin 22 to a resistor with a minimum resistance of 500 ohms. For a green LED circuit, you can use a 220-ohm resistor. This is because the frequency of red light is the highest in the visible light spectrum, and according to the following equation, energy is directly proportional to frequency:

$$\text{Energy} = h * \text{frequency}$$

This means that red LED bulbs heat up much faster than green LED bulbs. To prevent this, high-resistance resistors are needed. The other end of the resistor connects to the positive terminal of the LED bulb, marked by the LED's smaller leg. The negative end is connected to the GND (ground) pin of the Raspberry Pi Pico.

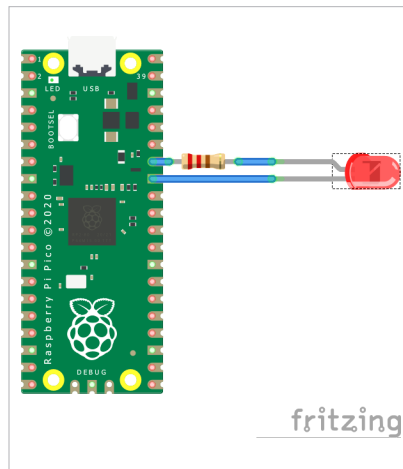


Figure 1: Raspberry Pi Pico with LED connected

Software configuration

To program the Raspberry Pi Pico with CircuitPython, follow these steps:

- Download the latest version of CircuitPython for Pico from the official website.
- Connect the Pico to the PC by holding down the BOOTSEL button while inserting the USB cable. This puts the Pico in bootloader mode, and a drive named RPI-RP2 will appear on the screen.
- Drag and drop the *adafruit_circuitpython_etc.uf2* file downloaded in step 1 to the RPI-RP2 drive. This installs CircuitPython on the Pico, and the drive disappears.
- A new drive called CIRCUITPY appears, which contains the files and folders that are required for CircuitPython to run on the board.

The main file we will use is *code.py*, which is the default CircuitPython code file executed when the board is powered on or reset. The *lib* folder contains the libraries or modules for our code.

Writing and running the code

To write and run our code, we can

use any text editor or IDE supporting CircuitPython. In this case, we will use Thonny IDE, a simple and beginner-friendly IDE with a built-in CircuitPython interpreter. To use Thonny IDE, we need to select the CircuitPython interpreter from the *Tools* menu and choose Raspberry Pi Pico as the device.

In Thonny IDE, we can write our code in the editor window and save it as *code.py* on the CIRCUITPY drive. For example, we can write code to blink an LED light connected to pin 22 of the Pico board with a 0.5 second interval, as shown here:

```
Blink.py
1 import time
2 import board
3 import digitalio
4
5 led = digitalio.DigitalInOut(board.
GP22)
6 led.direction = digitalio.Direction.
OUTPUT
7
8 while True:
9     led.value = not led.value
10    time.sleep(0.5)
11    led.value = not led.value
12    time.sleep(0.5)
13
```

In this article, we have learned how to program the Raspberry Pi Pico with CircuitPython. We covered downloading and installing CircuitPython, using Thonny IDE to write and run code, and demonstrated how to blink an LED light connected to the Pico board.

CircuitPython is a powerful and user-friendly programming language that empowers us to create a wide range of IoT projects using the Pico board and other hardware components. **END** 🐧

By: Anisha Ghosh

The author is a cybersecurity researcher and an active contributor to open source communities and repositories. She is interested in developing novel, scalable and secure systems.